

REQUISITOS

Requisitos del controller

- RF1:** El controller debe ser capaz de conectarse a un broker MQTT y suscribirse a los topics de los dispositivos para recibir sus mensajes.
- RF2:** El controller debe ser capaz de recibir enviar mensajes a los dispositivos a través de MQTT.
- RF3:** El controller debe ser capaz de recibir mensajes del bridge a través de un socket y responder a ellos.
- RF4:** El controller debe ser capaz de gestionar la persistencia del sistema.
- RF5:** El controller debe ser capaz de gestionar la lógica de las reglas del sistema, actualizándolo el estado de los dispositivos involucrados.
- RF6:** El controller debe ser capaz de gestionar la lógica de los dispositivos, actualizando su estado en función de los comandos del usuario.
- RF7:** El controller debe ser capaz de registrar y eliminar dispositivos

Requisitos del bridge

- RF8:** El bridge debe ser capaz de enviar y recibir mensajes con el controller a través de un socket.
- RF9:** El bridge debe ser capaz de enviar y recibir mensajes con el usuario a través de Discord.
- RF10:** El bridge debe ser capaz de conectarse con el bot de discord a través del token.

Requisitos de los dispositivos

- RF11:** Los dispositivos deben ser capaces de conectarse al broker MQTT y publicar su estado.
- RF12:** Los dispositivos deben ser capaces de recibir mensajes del controller a través de MQTT, procesarlos y responder a ellos.
- RF13:** Los dispositivos deben ser capaces de evaluar reglas y cambiar su estado en función de ellas.
- RF14:** Los dispositivos deben ser capaces de solicitar conectarse.

Requisitos del cliente

- RF15:** El cliente debe ser capaz de enviar mensajes al bridge a través de Discord.
- RF16:** El cliente debe ser capaz de recibir mensajes del bridge a través de Discord.

Requisitos del rule engine

- RF17** El rule engine debe ser capaz de gestionar la lógica de las reglas del sistema, actualizándolo el estado de los dispositivos involucrados mediante mensajes.

CASOS DE USO

A continuación se presentan casos de uso que reflejan el comportamiento de la aplicación ante diferentes escenarios.

Nombre de caso de uso: Pedir al bot que muestre la ayuda

- **Autor:** Antonio Morono
- **Actores involucrados:** bridge, bot, cliente, dispositivos

- **Resumen:** Un usuario accede al bot por primera vez y no sabe cuales son los comandos disponibles. El bot le muestra la ayuda.
- **Precondiciones:** El bot está activo y por tanto el bridge y el controller también. El cliente tiene el bot instalado en discord
- **Postcondiciones:** El bot muestra la ayuda en el chat
- **Flujo de eventos:** El bot recibe el comando `help` del cliente y el bridge responde al bot con la ayuda. El bot muestra la ayuda al cliente.
- **Clases involucradas:** `Bridge` aunque no es una clase como tal

Nombre de caso de uso: Pedir al bot que cambie el estado de un switch

- **Autor:** Antonio Morono
- **Actores involucrados:** bridge, controller, switch, bot, cliente
- **Resumen:** Un usuario escribe el comando `set <id> <estado>` en el chat del bot y el switch cambia de estado si todo va bien.
- **Precondiciones:** El bot está activo y por tanto el bridge y el controller también. El switch está registrado en el controller y el cliente tiene el bot instalado en discord
- **Postcondiciones:** El bot muestra la confirmación de que el switch ha cambiado de estado
- **Flujo de eventos:** El bot recibe el comando `set <id> <estado>` del cliente, el bridge manda el comando directamente al controller que es que lo gestiona. El controller envía el comando al switch y espera a que este lo confirme. El switch cambia de estado y responde al controller con el nuevo estado. El controller recibe la confirmación y manda la respuesta al bridge, que la muestra al cliente.
- **Clases involucradas:** `Bridge` aunque no es una clase como tal, `Controller`, `Switch`

Nombre de caso de uso: Pedir al bot que muestre el estado de un dispositivo

- **Autor:** Antonio Morono
- **Actores involucrados:** bridge, bot, cliente, dispositivos, controller
- **Resumen:** Un usuario accede al bot por primera vez y no sabe cuales son los comandos disponibles. El bot le muestra la ayuda.
- **Precondiciones:** El bot está activo y por tanto el bridge y el controller también. El switch está registrado en el controller y el cliente tiene el bot instalado en discord
- **Postcondiciones:** El bot muestra el estado del dispositivo en el chat
- **Flujo de eventos:** El bot recibe el comando `status <id>` del cliente y el bridge manda el comando directamente al controller que es que lo gestiona. El controller envía el comando al dispositivo para que muestre su estado y espera a que este lo confirme. El dispositivo responde al controller con el estado actual. El controller recibe la confirmación y manda la respuesta al bridge, que la muestra al cliente.
- **Clases involucradas:** `Bridge` aunque no es una clase como tal, `Controller`, `Switch` o `Sensor` o `Clock`

Nombre de caso de uso: Introducir una regla

- **Autor:** Antonio Morono
- **Actores involucrados:** bridge, bot, cliente, dispositivos, controller, rule engine
- **Resumen:** El usuario establece una regla con la sintaxis definida y la lista de reglas se actualiza, modificando el comportamiento del sistema.

- **Precondiciones:** El bot está activo y por tanto el bridge y el controller también. El cliente tiene el bot instalado en discord
- **Postcondiciones:** El bot muestra que la regla se ha añadido correctamente.
- **Flujo de eventos:** El bot recibe el comando `add_rule <rule>` del cliente y el bridge manda el comando directamente al controller que es que lo gestiona. El controller envía la regla al rule engine, que la añade a su lista de reglas. El rule engine responde al controller con la confirmación de que la regla se ha añadido correctamente. El controller manda la respuesta al bridge, que la muestra al cliente.
- **Clases involucradas:** `Bridge` aunque no es una clase como tal, `Controller`, `Rule Engine`, `Switch` o `Sensor` o `Clock`

CLASES PRINCIPALES Y RELACIÓN ENTRE ELLAS

- **Controller:** Es el núcleo del sistema. Se encarga de gestionar la comunicación con los dispositivos mediante mqtt, mantener el estado y tipo de cada dispositivo, procesar los comandos recibidos desde el bridge (bot de Discord) y aplicar las reglas definidas en el sistema. Además, se ocupa de la persistencia del estado y de la interacción con el rule engine.
- **RuleEngine:** Permite añadir, editar, eliminar y listar reglas, así como evaluarlas cada vez que cambia el estado de un dispositivo. Las reglas se almacenan de forma persistente y se reconstruyen al iniciar el sistema.
- **Database:** Proporciona funciones para guardar y cargar el estado del sistema y las reglas en archivos binarios utilizando pickle. Así, logramos la persistencia del estado entre ejecuciones.
- **Bridge:** Es el bot de Discord que actúa como interfaz entre el usuario y el sistema. Recibe comandos de los usuarios, los envía al controller a través de un socket y muestra las respuestas recibidas. Permite el control remoto y la consulta del estado del sistema de forma sencilla y accesible.
- **Sensor, Switchy Clock:** Simulan dispositivos IoT. El Sensor publica periódicamente su estado y responde a peticiones del controller. El Switch puede cambiar de estado bajo demanda y confirma los cambios al controller. El clock publica la hora periódicamente y responde a peticiones del controller.

Relación entre ellas

El bridge recibe comandos del usuario y los transmite al controller. El controller procesa estos comandos, consulta o modifica el estado de los dispositivos (Sensor, Switch, etc.) a través de mqtt y aplica las reglas definidas en el rule engine. El estado y la configuración del sistema se guardan y cargan usando la clase database. Así, todos los componentes colaboran para ofrecer un sistema domótico modular, robusto y fácilmente ampliable.

DECISIONES DE DISEÑO E IMPLEMENTACIÓN

1. ¿Controller y Rule engine han de ser aplicaciones separadas?, ¿por qué?, ¿qué ventajas tiene una y otra opción?

Hemos decidido que controller y rule engine formen parte de la misma aplicación.

Por un lado, no vemos ventajas a que sean aplicaciones separadas, ya que ambas deben estar funcionando para que el sistema pueda ejecutarse correctamente, y no se nos ocurre un escenario en el que pudiese ser conveniente correrlas en procesos o incluso dispositivos separados. Y tal como lo hemos implementado, el sistema cumple con la funcionalidad solicitada.

Por otro lado, es evidentemente más simple implementarlas como parte de una misma aplicación, y elimina tiempos de comunicación innecesarios entre procesos.

2. ¿Cómo se comunican Controller y Rule engine en la opción escogida?

Puesto que ambos son clases del mismo proceso, se comunican directamente llamando a las funciones pertinentes de la otra clase.

De esta forma obtenemos una comunicación simple, efectiva y poco propensa a errores.

3. ¿Tiene sentido que alguno de estos componentes compartan funcionalidad?, ¿qué relación hay entre ellos?

Controller y Rule Engine están estrechamente relacionados, ya que ambos gestionan el estado y la lógica de los dispositivos. Sin embargo, sus responsabilidades están claramente separadas:

- **Controller** se encarga de la comunicación con los dispositivos, la gestión de estados y la interacción con el usuario/bot.
- **Rule Engine** se encarga exclusivamente de la gestión y evaluación de reglas automáticas sobre los dispositivos.

No es conveniente que compartan funcionalidad más allá de estructuras de datos básicas (como los diccionarios de estado de dispositivos), ya que cada uno debe estar desacoplado para facilitar el mantenimiento y la escalabilidad.

La relación entre ellos es de colaboración: el Controller consulta y aplica las reglas gestionadas por el Rule Engine, pero cada uno mantiene su propia lógica y responsabilidades.

Esta separación permite modificar o ampliar el motor de reglas sin afectar al controlador, y viceversa.

4. ¿Cuántas instancias hay de cada componente?

- En cuanto a **bridge**, está pensado para que haya una única instancia. Cualquier persona que se comunique con el bot lo estará haciendo con ese bridge, por lo que nuestro sistema ya admite que varias personas lo usen simultáneamente, así que es la decisión natural.
- **Controller/Rule engine** por su parte también está pensado para ser único. Puede comunicarse con todos los dispositivos que sea necesario, y el sistema es para una única casa, así que no vemos necesario que haya más de un controller.

5. ¿Controller y Bridge han de ser aplicaciones separadas?, ¿por qué?, ¿qué ventajas tiene una y otra opción?

Sí, consideramos conveniente que controller y bridge sean aplicaciones separadas.

Implementarlo apenas supone un mayor esfuerzo, ya que los sockets facilitan mucho la comunicación, y a cambio podemos desplegar ambas partes en diferentes servidores. Además, así evitamos que bridge se bloquee y deje de responder ante una petición de ayuda o incorrecta si el rule engine está ocupado.

6. ¿Cómo se comunican Controller y Bridge en la opción escogida?

Controller y bridge se comunican utilizando **sockets**. Hemos elegido esta opción porque es sencilla de implementar, la hemos utilizado previamente por lo que tenemos cierta familiaridad, y se ajusta perfectamente al esquema petición-respuesta que hemos implementado.

ESQUEMA DE DATOS PARA LA PERSISTENCIA

El sistema guarda su estado en archivos binarios utilizando la clase `database.py`, que utiliza a su vez el módulo `pickle`. Hemos reutilizado, con pequeñas modificaciones, el código de la práctica anterior, ya que nos dio muy buenos resultados. Los datos persistentes son:

- **Estado de los dispositivos.** Un diccionario que utiliza como clave los ids de los dispositivos y como valor el último estado registrado.
- **Tipo de los dispositivos.** Un diccionario que utiliza como clave los ids de los dispositivos y como valor el tipo de dispositivo (por ejemplo, `boiler_switch`).
- **Reglas.** Debido a que `pickle` no puede guardar expresiones lambda, hemos optado por almacenar en la estructura de datos de las reglas su versión en string y su id (esto, por otro lado, nos facilita enormemente la implementación del listado de reglas). Dicho string también se ajusta en caso de edición de reglas. De este modo, al inicializar el controller cargamos las versiones de texto de las reglas y reconstruimos toda la estructura de datos.
- **Último id de reglas.** Rule engine utiliza parte del id 0, y con cada regla creada se aumenta en 1 y se le asigna dicho valor. Por lo tanto, es fundamental almacenar dicho valor. Esto nos permite asegurar que dos reglas jamás tendrán el mismo valor. Por ejemplo, si la regla con mayor id tiene id 7 y la borramos, la siguiente que creamos tendrá id 8. Creemos que así prevenimos confusiones.

LIMITACIONES

Como ya se ha comentado en la sección de decisiones de diseño, el sistema acepta únicamente una instancia de cada componente. No obstante, más que una limitación lo consideramos una característica propia del tipo de sistema que se solicita (domótico).

CONCLUSIONES

En conclusión, hemos aprendido a utilizar `mqtt` y reforzado nuestros conocimientos sobre este tipo de sistemas.

También hemos aprendido a montar los bots de discord, algo que nos ha resultado especialmente interesante por ser una herramienta que utilizamos con frecuencia incluso fuera de ambiente universitario. Y la posibilidad de crear bots con otros propósitos es genial.

También es interesante destacar el sistema de confirmación de estados, ya comentado, que nos permite tener certeza de que los dispositivos que nos contesten lo hacen a la petición indicada. Puesto que no se solicitaba explícitamente en el enunciado, suponemos que puede considerarse una funcionalidad "extra".

En definitiva, estamos contentos con lo que hemos aprendido y con el sistema que hemos desarrollado, ya que nos parece robusto, eficiente y modular.